VUPTH

Product Requirements Document

APP PARCEIRO — Frontend

Versão	Data	Status	Backend
v14	Mai 2026	Em desenvolvimento	v14 – 120 testes ✓

1. Visão Geral do Produto

1.1 Problema e Proposta de Valor

O VUPTH elimina as filas físicas em estabelecimentos de beleza e saúde. O App Parceiro é a ferramenta de gestão para donos de estabelecimento (Owners) e profissionais (Professionals) — permite criar e administrar o negócio, gerenciar a fila em tempo real e controlar atendimentos.

1.2 Personas

Persona	Descrição
Owner (Dono)	Cria o estabelecimento, cadastra profissionais, define serviços e abre/fecha a loja. Pode ser também profissional da própria loja (solo shop).
Professional	Gerencia sua própria fila: inicia, pausa e finaliza atendimentos. Adiciona guests manualmente. Pode ter sido adicionado por um Owner ou ter criado seu próprio estabelecimento.
Solo Shop	Owner que também é profissional da própria loja. Passa pelos dois fluxos de onboarding sequencialmente.

1.3 Stack Frontend

Tecnologia	Detalhe
Framework	React Native + Expo SDK 51
Navegação	Expo Router v3 (file-based)
UI	Gluestack UI v1 + NativeWind v4

Tecnologia	Detalhe
Estado global	Zustand v4
Dados servidor	TanStack React Query v5 (polling 30s)
Formulários	React Hook Form + Zod
HTTP	Axios com interceptor de refresh automático
Storage seguro	Expo SecureStore (tokens JWT)
Tipografia	DM Sans (corpo) + DM Mono (números/tempos)

1.4 Paleta de Cores

Token	Hex	Uso	Tailwind
`primary`	#1B4332	Fundo principal, headers, textos dominantes	`bg-primary`
`accent`	#52B788	Ícones, bordas ativas, elementos secundários	`bg-accent`
`cta`	#00B37E	Botões de ação primária (CTA)	`bg-cta`
`bg`	#F7FAF8	Background de telas	`bg-bg`
`muted`	#8A9E94	Texto secundário, placeholders	`text-muted`
`danger`	#E53E3E	Erros, cancelamentos	`text-danger`

2. Autenticação e Sessão

2.1 Sistema de Tokens

O backend usa JWT RS256. O access_token expira em 1 hora. O refresh_token dura 7 dias e é rotativo — cada uso gera um novo par. Ambos devem ser armazenados no Expo SecureStore (nunca em AsyncStorage).

Item	Detalhe
Access token TTL	1 hora
Refresh token TTL	7 dias
Rotação	Cada POST /auth/refresh emite novo par — salvar ambos
Storage	Expo SecureStore — chaves: vupth_access_token, vupth_refresh_token

Item	Detalhe
Interceptor	Axios intercepta 401, faz refresh automático e repete a request original

2.2 Sistema de Roles

Todo usuário nasce com a role CLIENT. Roles adicionais são concedidas ao usar endpoints específicos. CRÍTICO: após ganhar uma nova role, o frontend deve chamar POST /auth/refresh para obter um token com a role atualizada — sem isso, os endpoints retornarão 403.

Role	Como é concedida	Endpoint que concede
CLIENT	Automática no registro	POST /auth/register
OWNER	Ao criar establishment	POST /establishment/quick-create
PROFESSIONAL	Ao criar perfil profissional	POST /professional/quick-create ou /professional/register

> ⚠️ *Após POST /establishment/quick-create → OBRIGATÓRIO chamar POST /auth/refresh antes de qualquer endpoint OWNER.*
>
> ⚠️ *Após POST /professional/quick-create → OBRIGATÓRIO chamar POST /auth/refresh antes de qualquer endpoint PROFESSIONAL.*

2.3 Payload do JWT

O frontend pode decodificar o payload (base64 da segunda parte do token) sem chamar o backend:

```
{ "sub": "uuid-do-usuario", "email": "user@email.com", "roles": ["CLIENT",
"OWNER"], "exp": 1234567890 }
```

Usar getRolesFromToken(token) do arquivo src/lib/jwt.ts para extrair as roles e redirecionar o usuário ao fluxo correto.

3. Fluxos de Tela

3.1 Onboarding — Escolha de Papel

Tela inicial do app parceiro. O usuário escolhe seu papel antes de criar conta:

Opção	Destino
Sou dono de estabelecimento	Fluxo Owner → register → quick-create establishment → quick-create professional (opcional) → dashboard
Sou profissional	Fluxo Professional → register → professional/register → tela de aguardo de vinculação

3.2 Fluxo Owner (Dono)

Sequência obrigatória de endpoints para completar o onboarding:

#	Endpoint	O que salvar
1	POST /auth/register	— (auto-login na sequência)
2	POST /auth/login	access_token, refresh_token, user.id
3	POST /establishment/quick-create	establishment.id, establishment.name
4	POST /auth/refresh ⚠️	Novo access_token (agora inclui OWNER)
5	POST /establishments/{id}/addresses	—
6	POST /establishment/{id}/service-items	service_item.id (salvar para criar attendances)
7	Se solo shop: POST /professional/quick-create	professional.id
8	Se solo shop: POST /auth/refresh ⚠️	Novo access_token (agora inclui PROFESSIONAL)

3.3 Fluxo Professional (sem establishment próprio)

#	Endpoint	O que salvar
1	POST /auth/register	—
2	POST /auth/login	access_token, refresh_token, user.id
3	POST /professional/register	professional.id
4	POST /auth/refresh ⚠️	Novo access_token (agora inclui PROFESSIONAL)
5	POST /professional/affiliate (opcional)	Afiliar a um establishment existente

3.4 Dashboard do Profissional

Tela principal após onboarding. Mostra a fila em tempo real via polling de 30s:

- GET /establishment/id/{id}/professionals-with-queue → lista de profissionais com filas
- Profissional vê sua própria fila completa com nomes e quantidade
- PATCH /attendances/{id}/start → iniciar atendimento do próximo da fila
- PATCH /attendances/{id}/finish → finalizar atendimento atual
- PATCH /attendances/{id}/pause → pausar atendimento (status → PAUSED)
- PATCH /attendances/{id}/resume → retomar atendimento pausado
- DELETE /professional-queue/{id}/remove → remover cliente da fila
- POST /professional-queue/add-quest-attendance → adicionar guest manualmente

4. Referência de Endpoints — App Parceiro

4.1 Auth — /auth

Método	Endpoint	Auth	HTTP
POST	/auth/register	Público	201
POST	/auth/login	Público	200
POST	/auth/refresh	Público	200
POST	/auth/logout	Bearer	204

POST /auth/register — Body:

```
{ "email": "user@email.com", "password": "Senha@123" }
```

POST /auth/login — Response tokens:

```
{ "user": {...}, "tokens": { "access_token": "...", "refresh_token": "...",
"expires_in": 604799 } }
```

POST /auth/refresh — Body:

```
{ "refresh_token": "..." }
```

⚠️ *O refresh é ROTATIVO. Salvar sempre o novo refresh_token retornado. O antigo é invalidado.*

4.2 Establishment — /establishment

Método	Endpoint	Auth	HTTP
POST	/establishment/quick-create	Bearer CLIENT	201
GET	/establishment	Público	200
GET	/establishment/id/{id}	Público	200
GET	/establishment/id/{id}/professionals-with-queue	Público	200
PATCH	/establishment/{id}/open-status	Bearer OWNER	200
POST	/establishment/{id}/service-items	Bearer OWNER	201
POST	/establishment/{id}/professionals	Bearer OWNER	201

POST /establishment/quick-create — Body:

```
{ "name": "Barbearia Top", "serviceType": "BARBER" }
```

POST /establishment/{id}/service-items — Body:

```
{ "serviceItemType": "CORTE_MAQUINA", "customName": "Corte Degradê", "price":
35.00, "estimatedMinutes": 30 }
```

POST /establishment/{id}/professionals — Body (Owner adiciona prof existente):

```
{ "professionalId": "uuid-do-profissional" }
```

GET /establishment/id/{id}/professionals-with-queue — Response (objeto professionals[]):

```
{ "id": "...", "name": "...", "specialty": "...", "is_available": true,
"attendancesInQueue": [...], "estimated_wait_minutes": 15 }
```

4.3 Professional — /professional

Método	Endpoint	Auth	HTTP
POST	/professional/quick-create	Bearer CLIENT	201
POST	/professional/register	Bearer CLIENT	201
POST	/professional/affiliate	Bearer PROFESSIONAL	200
GET	/professional/{id}/service-items	Público	200

POST /professional/quick-create (solo shop — cria e vincula ao establishment):

```
{ "name": "João Silva", "establishmentId": "uuid", "specialty": "BARBER" }
```

POST /professional/register (profissional sem establishment):

```
{ "name": "João Silva", "specialty": "BARBER" }
```

POST /professional/affiliate (vincula profissional a um establishment):

```
{ "establishmentId": "uuid" }
```

⚠️ *MVP: profissional só pode estar vinculado a um establishment por vez. Tentar afiliar novamente → 409 PROFESSIONAL_ALREADY_AFFILIATED.*

4.4 Attendances — /attendances

Método	Endpoint	Auth	HTTP
GET	/attendances/{id}	Bearer (any)	200
POST	/attendances	Bearer CLIENT	201
PUT	/attendances/{id}/cancel	Bearer CLIENT	200
DELETE	/attendances/{id}/leave-queue	Bearer CLIENT	200
PATCH	/attendances/{id}/start	Bearer PROFESSIONAL	200
PATCH	/attendances/{id}/finish	Bearer PROFESSIONAL	200
PATCH	/attendances/{id}/pause	Bearer PROFESSIONAL	200
PATCH	/attendances/{id}/resume	Bearer PROFESSIONAL	200

⚠️ *POST /attendances: clientId no body é IGNORADO pelo backend — sempre usa o do JWT. Não exibir campo clientId no formulário.*

⚠️ *Cliente só pode cancelar attendances com status SCHEDULED. IN_PROGRESS, PAUSED e FINISHED → 403.*

4.5 Professional Queue — /professional-queue

Método	Endpoint	Auth	HTTP
POST	/professional-queue/add-quest-attendance	Público	201
DELETE	/professional-queue/{id}/remove	Bearer PROFESSIONAL	200

POST /professional-queue/add-quest-attendance — Body:

```
{ "establishmentId": "uuid", "professionalId": "uuid", "serviceItemIds": ["uuid"],
"clientName": "José (walk-in)" }
```

4.6 Address — /establishments e /users/me

Método	Endpoint	Auth	HTTP
POST	/establishments/{id}/addresses	Bearer OWNER	201
POST	/users/me/addresses	Bearer	201
GET	/users/me/addresses	Bearer	200
GET	/users/me/addresses/main	Bearer	200
PATCH	/users/me/addresses/{id}	Bearer	200
PATCH	/users/me/addresses/{id}/main	Bearer	200
DELETE	/users/me/addresses/{id}	Bearer	200

zipCode: sempre 8 dígitos puros, sem hífen — ex: 01310100. Frontend formata para exibição: 01310-100.

4.7 Profile — /profile

Método	Endpoint	Auth	HTTP
PATCH	/profile/general-infos	Bearer	200
PATCH	/profile/update-email/me	Bearer	200
PATCH	/profile/update-password/me	Bearer	200
DELETE	/profile/delete/me	Bearer	204

PATCH /profile/general-infos — Todos os campos opcionais (PATCH semântico). Body vazio → 400 NO_FIELDS_TO_UPDATE:

```
{ "first_name": "João", "last_name": "Silva", "phone": "11999999999", "birth_date":
"1990-05-15", "gender": "MALE", "profile_image_url": "https://..." }
```

⚠️ *Campo desconhecido no body (ex: 'name') → 400 UNKNOWN_PROFILE_FIELD. Usar apenas os campos documentados acima.*

5. Enums da API

5.1 ServiceType — tipo do establishment

Valor	Descrição
BARBER	Barbearia

Valor	Descrição
SALON_BEAUTY	Salão de beleza
EYEBROW	Design de sobrancelha
MANICURE	Manicure / pedicure

5.2 ServiceItemType — serviços oferecidos

Valor	Descrição
CORTE_MAQUINA	Corte na máquina (BARBER)
RECORTE_PONTAS	Recorte das pontas (SALON_BEAUTY)
PINTURA	Pintura (BARBER, SALON_BEAUTY)
MANICURE	Manicure (MANICURE)
PEDICURE	Pedicure (MANICURE)

5.3 AttendanceStatus

Valor	Quando ocorre
SCHEDULED	Na fila, aguardando ser chamado
IN_PROGRESS	Sendo atendido agora
PAUSED	Atendimento pausado temporariamente
FINISHED	Atendimento concluído
CANCELLED	Cancelado pelo cliente ou removido pelo profissional (duplo L)

5.4 OpenStatus — estado do establishment

Valor	Descrição
OPEN	Aberto e recebendo clientes
CLOSED	Fechado
TEMPORARILY_CLOSED	Fechado temporariamente

6. Regras de Negócio — Frontend

6.1 Tokens e Roles

- Sempre renovar o token após ganhar nova role (OWNER ou PROFESSIONAL)

- Interceptor Axios: ao receber 401, tentar refresh automático 1x antes de redirecionar ao login
- Tokens armazenados no Expo SecureStore — nunca AsyncStorage ou memória
- Após logout: refresh_token é invalidado no servidor; access_token expira naturalmente
- O JWT payload contém: sub (userId), email, roles, exp — decodificar client-side para routing

6.2 Attendances

- Não enviar clientId no body de POST /attendances — backend usa o do JWT
- Cliente só pode cancelar attendances SCHEDULED — mostrar botão cancelar apenas se status === SCHEDULED
- Profissional inativo (is_available: false) não aparece na listagem do cliente
- Seleção de 'menor fila': frontend ordena por estimated_wait_minutes

6.3 Fila e Polling

- Polling de 30s via React Query para atualizar filas em tempo real
- Cliente não vê quantidade de pessoas na fila, só tempo estimado
- Profissional vê lista completa com nomes e posições
- estimated_wait_minutes é sempre estimado — exibir como '~15 min', nunca como promessa

6.4 zipCode

- API recebe e armazena 8 dígitos puros: 01310100
- Frontend formata para exibição: 01310-100
- Frontend deve remover o hífen antes de enviar para a API

6.5 Datas

- Todos os campos de data da API são ISO-8601 com offset: 2026-05-02T10:00:00Z
- Enviar scheduledTime sempre com offset — nunca como timestamp numérico
- Backend opera em UTC — considerar fuso do usuário ao exibir horários

7. Tratamento de Erros

Todos os erros seguem a mesma estrutura de resposta:

```
{ "status": 422, "message": "Field validation error", "code": "VALIDATION_ERROR",
"fieldErrors": { "email": "must be a valid email" }, "timestamp": "..." }
```

HTTP	code	Quando tratar
400	INVALID_REQUEST_BODY	UUID mal formado, tipo errado, campo unknown no profile

HTTP	code	Quando tratar
400	NO_FIELDS_TO_UPDATE	Body vazio em PATCH /profile/general-infos
400	UNKNOWN_PROFILE_FIELD	Campo não reconhecido no body de update profile
401	—	Token ausente, inválido ou expirado → tentar refresh, senão → login
403	—	Autenticado mas sem role necessária → não mostrar o recurso
404	—	Recurso não encontrado → mostrar estado vazio
409	PROFESSIONAL_ALREADY_AFFILIATED	Profissional já tem establishment — MVP: 1 por vez
422	VALIDATION_ERROR	Checar fieldErrors para mostrar erro no campo correto

8. Estrutura de Pastas Recomendada

```
vupth-partner/
├── app/                          ← Expo Router (file-based routing)
│   ├── (auth)/                   ← Grupo sem tab bar
│   │   ├── index.tsx             ← Tela de escolha de papel
│   │   ├── login.tsx
│   │   └── register.tsx
│   ├── (owner)/                  ← Fluxo Owner
│   │   ├── onboarding/
│   │   │   ├── establishment.tsx  ← Formulário quick-create
│   │   │   ├── address.tsx
│   │   │   └── services.tsx
│   │   └── dashboard.tsx         ← Fila em tempo real
│   ├── (professional)/           ← Fluxo Professional
│   │   ├── onboarding.tsx        ← Register + affiliate
│   │   └── queue.tsx             ← Fila do profissional
│   └── _layout.tsx
├── src/
│   ├── lib/
│   │   ├── api.ts               ← Axios + interceptor de refresh
│   │   └── jwt.ts               ← Decode payload, getRolesFromToken
│   ├── store/
│   │   └── auth.store.ts        ← Zustand: user, tokens, login/logout
│   ├── hooks/
│   │   ├── useEstablishment.ts  ← React Query para establishment
│   │   └── useQueue.ts          ← React Query com polling 30s
│   └── global.css
```

```
├── tailwind.config.js
├── babel.config.js
└── metro.config.js
```